VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Machhe Belagavi - 590018

Major Project Phase - II Report

on

LANE-GUARDIAN:Impact of Lane Discipline Violations on Saturation Flow Rate at Signalized Intersections

Course Code: CSEP23706



Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Engineering

in

Computer Science and Engineering

Submitted by

Abhigna M N	**1GA23CS001**
Divya H	**1GA23CS043**
Niveditha K U	**1GA23CS113**
Hanumantha S	**1GA24CS403**

Under the Guidance of

Mr R C Ravindranath

Assistant Professor

GLOBAL ACADEMY OF TECHNOLOGY
growing ahead of time
Autonomous Institute, Affiliated to VTU

Department of Computer Science and Engineering

Global Academy of Technology

Rajarajeshwari Nagar, Bengaluru - 560 098

2026-2027

GLOBAL ACADEMY OF TECHNOLOGY

(An Autonomous Institution Affiliated to VTU, Belagavi)
Ideal Homes Township, Rajarajeshwari Nagar, Bengaluru - 560098
Department of Computer Science and Engineering

CERTIFICATE

Certified that the Major Project Phase-II work entitled **LANE-GUARDIAN: Impact of Lane Discipline Violations on Saturation Flow Rate at Signalized Intersections** has been successfully carried out at **Global Academy of Technology** by **Abhigna M N** bearing **1GA23CS001**, **Divya H** bearing **1GA23CS043**, **Niveditha K U** bearing **1GA23CS113** and **Hanumantha S** bearing **1GA24CS403** bonafide students of **Global Academy of Technology** in partial fulfillment of the requirements of final year degree in **Bachelor of Engineering in Computer Science and Engineering** during Academic Year **2026-27**. The Major Project Phase-II report has been approved as it satisfies the academic requirements in respect of project work for the said degree.

Mr. R C Ravindranath Project Guide Assistant Professor Dept. of CSE, GAT	**Ms. Supriya H S** Project Coordinator Assistant Professor Dept.of CSE, GAT
Dr. R Rajkumar Professor and Head Dept. of CSE, GAT	**Dr. S B Bhanu Prashanth** Principal GAT

EXTERNAL VIVA

Name of the Examiners	**Signature with Date**
1.	
2.	

GLOBAL ACADEMY OF TECHNOLOGY
(An Autonomous Institution Affiliated to VTU, Belagavi)
Ideal Homes Township, Rajarajeshwari Nagar, Bengaluru - 560098
Department of Computer Science and Engineering

DECLARATION

We, **Abhigna M N** bearing **1GA23CS001**, **Divya H** bearing **1GA23CS043**, **Niveditha K U** bearing **1GA23CS113** and **Hanumantha S** bearing **1GA24CS403** students of VII Semester B.E, Department of Computer Science and Engineering, Global Academy of Technology, Rajarajeshwari Nagar, Bengaluru, declare that the Major Project Work Phase-II entitled **LANE-GUARDIAN: Impact of Lane Discipline Violations on Saturation flow Rate at Signalized Intersections** has been carried out by us and submitted in partial fulfillment of the requirements for the award of degree in Bachelor of Engineering in Computer Science and Engineering from Visvesvaraya Technological University, Belagavi during the Academic Year 2026 – 2027.

Abhigna M N	1GA23CS001
Divya H	1GA23CS043
Niveditha K U	1GA23CS113
Hanumantha S	1GA24CS403

Place: Bengaluru

Date:

ABSTRACT

Indian cities suffer from congested roads, dense and mixed traffic, and poor lane discipline. Traffic is usually monitored by a policeman on the spot or by costly loop detectors and radar, which report overall flow but nothing about how individual vehicles behave within a lane. LANE-GUARDIAN is a camera-based system that analyses recorded fixed-camera videos of major Indian intersections, with no roadside sensors. It detects, classifies and tracks vehicles, detects lane discipline violations, generates fines and estimates lane-wise flow. Vehicles are detected with a custom-trained YOLOv10n model for five classes (car, two-wheeler, auto-rickshaw, bus and truck), which reaches 87.68 mAP@0.5 on the validation split of the Indian Road User Vehicle Dataset (IRUVD), and ByteTrack keeps each vehicle's ID through short misses. Lane geometry comes from the vanishing point, an inverse perspective transform, road markings and vehicle trajectories, with temporal stabilisation and camera-motion correction, and lanes can be merged or split manually if automatic estimation fails. Vehicles crossing a virtual line are counted to give lane flow in vehicles and PCU per hour, the maximum 60-second flow, and an indicative Level of Service from mean speed. The outputs are an annotated video, a multi-sheet Excel report, an HTML dashboard and a review page where an analyst checks each violation against its evidence image, confirms or rejects it, and edits the suggested fine, so the final report records the human decision. The whole pipeline runs through a web interface in Google Colab, so no coding is needed. It was tested end-to-end on published intersection and roadway recordings and checked.

ACKNOWLEDGMENT

The satisfaction and the euphoria that accompany the successful completion of any task would be incomplete without the mention of the people who made it possible. The constant guidance of these persons and encouragement provided, crowned our efforts with success and glory. Although it is not possible to thank all the members who helped for the completion of the Project Phase-II individually, we take this opportunity to express our gratitude to one and all.

We are grateful to the Management and our Institute **Global Academy of Technology** with its very ideals and inspiration for having provided us with the facilities, which made this, Project Phase-II a success.

We express our sincere gratitude to our esteemed Principal, **Dr. S B Bhanu Prashanth**, for his visionary leadership, constant support and encouragement, which have provided us with an excellent environment for learning and growth.

We also place on record our heartfelt thanks to **Dr. R. Rajkumar**, Professor and Head, Department of Computer Science and Engineering, for his invaluable guidance, continuous encouragement, and unwavering support throughout the course of this work. His motivation and inspiration have greatly contributed to the successful completion of our project.

We are deeply grateful to our Guide **Mr R. C. Ravindranath**, Assistant Professor Department of CSE, for the valuable guidance, encouragement, and suggestions provided throughout the project.

We also thank the Project Coordinator **Ms. Supriya H S**, Assistant Professor along with all the faculty members and staff of the Department of CSE for providing relevant information and helping in different capacities in carrying out Major Project Work.

Finally, we express our sincere thanks to our parents, friends, and well-wishers who directly or indirectly supported us in completing this project successfully.

Abhigna M N	1GA23CS001
Divya H	1GA23CS043
Niveditha K U	1GA23CS113
Hanumantha S	1GA24CS403

Justification for Mapping SDG 11 to the Project

This project aligns with SDG 11 (Sustainable Cities and Communities) by leveraging computer vision and multi-object tracking to address urban traffic congestion and poor lane discipline at signalized intersections. The solution promotes sustainable urban mobility by developing an automated pipeline using YOLOv10n and ByteTrack to detect lane violations and measure their direct impact on saturation flow rates. By providing a cost-effective, software-only tool that utilizes existing CCTV feeds, the project empowers traffic authorities and city planners to optimize road networks without requiring expensive physical sensors. This contributes to making city roads safer, alleviating congestion, and maximizing the efficiency of existing urban transport infrastructure, which are core objectives of SDG 11.

SDG Contribution Statement:

- Promotes safe and sustainable urban mobility by monitoring and quantifying lane discipline violations in mixed traffic.
- Strengthens urban transport infrastructure by providing actionable data, such as Level of Service and saturation flow, for traffic optimization.
- Enhances road safety by identifying high-risk behaviors such as wrong-way driving, overspeeding, and unsafe lane changes.
- Supports cost-effective smart city initiatives by utilizing existing fixed-angle traffic camera feeds instead of requiring expensive new hardware installations.
- Encourages data-driven urban planning to reduce congestion and improve the efficiency of city intersections.

Signature of the Student **Signature of the Project Guide**

CONTENTS

ABSTRACT **i**

ACKNOWLEDGMENT **ii**

List of Figures **vii**

List of Tables **viii**

GLOSSARY **ix**

1 INTRODUCTION **1**

1.1 Problem Definition 2

1.2 Objectives 2

1.3 Scope of the Project Work 3

1.4 Organization of the Report 4

2 REVIEW OF LITERATURE **5**

2.1 System Study 5

2.1.1 Overview of Existing Work 5

2.1.2 Research Gaps 7

2.2 Proposed Work 7

3 SYSTEM REQUIREMENT SPECIFICATION **8**

3.1 Functional Requirements 8

3.2 Non-Functional Requirements 9

3.3 Hardware Requirements 9

3.4 Software Requirements 10

4 SYSTEM DESIGN **11**

4.1 Design Overview 11

4.2 System Architecture 12

4.3 Data Flow Diagrams 13

4.3.1 Data Flow Diagram - Level 0 . . . 13
4.3.2 Data Flow Diagram - Level 1 . . . 14
4.3.3 Data Flow Diagram - Level 2 . . . 14
4.4 Use Case Diagram . . . 15
4.5 Sequence Diagram . . . 16
4.6 Modules of the Project . . . 17
4.6.1 Module 1: Video Preprocessing and Camera Calibration . . . 17
4.6.2 Module 2: Vehicle Detection and Classification . . . 17
4.6.3 Module 3: Multi-Object Tracking . . . 18
4.6.4 Module 4: Lane Discipline Violation Engine . . . 18
4.6.5 Module 5: Traffic Flow and Observed-Flow Analysis . . . 18
4.6.6 Module 6: Reporting, Review, Dashboard, and Web Interface . . . 19

5 IMPLEMENTATION 20

5.1 Steps for Implementation . . . 20
5.1.1 Installation of Hardware and Software Utilities . . . 20
5.1.2 Sample Data Used . . . 21
5.1.3 Debugging Phase . . . 21
5.1.4 Implementation Steps . . . 22
5.2 Implementation Issues . . . 23
5.3 Algorithms . . . 24

6 TESTING 28

6.1 Test Environment . . . 28
6.2 Unit Testing . . . 29
6.3 Integration Testing . . . 29
6.4 System Testing . . . 30
6.5 Functional Testing . . . 31

7 RESULTS AND DISCUSSION 32

7.1 Interface Screens . . . 32
7.2 Calibration, Detection and Tracking . . . 34

7.3 Flow Analysis 35
7.4 Violations 36
7.5 Dashboard and Reports 38

8 CONCLUSION AND FUTURE SCOPE **40**
8.1 Conclusion 40
8.2 Future Scope 40

REFERENCES **41**

References **41**

Appendix **44**
i) Plagiarism Report 44
ii) Conference/Journal Paper and Certificate 44
iii) Project Exhibition Certificate 44

LIST OF FIGURES

4.1 System Architecture 12

4.2 Data Flow Diagram - Level 0 13

4.3 Data Flow Diagram - Level 1 14

4.4 Data Flow Diagram - Level 2 15

4.5 Use Case Diagram 15

4.6 Sequence Diagram 16

7.1 Gradio web interface: video upload and calibration engine selection 33

7.2 Road view with calibrated lane overlay before analysis 33

7.3 Manual lane addition screen 34

7.4 Calibration diagnostics – vanishing point and fitted lane boundaries 34

7.5 Annotated output frame with tracked vehicles and lane overlays 35

7.6 Lane-wise flow, PCU/hour, and Level of Service 36

7.7 Vehicle class distribution and PCU contribution 36

7.8 Violation log – type, severity, lane, and confidence 37

7.9 Evidence crop for a flagged Lane Straddling violation 37

7.10 Violation logic and threshold settings page 38

7.11 Review & Fines tab – evidence, suggested fine, and approval 38

7.12 Dashboard summary view 39

7.13 Excel report with the Summary and Violations sheets 39

LIST OF TABLES

2.1 Literature Survey Table 5

3.1 Functional requirements 8

3.2 Non Functional Requirements 9

3.3 Hardware Requirement Specification 9

3.4 Software Requirement Specification 10

6.1 Unit Testing 29

6.2 Integration Testing 29

6.3 System Testing 30

6.4 Functional Testing 31

GLOSSARY

Sl. No.	Term	Full Form	Meaning
1	**PCU**	Passenger Car Unit	Weight that converts each vehicle type into car equivalents.
2	**LOS**	Level of Service	Traffic-condition class from A (free flow) to F (congested).
3	**YOLO**	You Only Look Once	Single-stage object detector used to find vehicles.
4	**NMS**	Non-Maximum Suppression	Removes duplicate detection boxes; not needed in YOLOv10.
5	**IoU**	Intersection over Union	Overlap between two boxes, used to match and score boxes.
6	**mAP**	Mean Average Precision	Overall accuracy measure of the detector.
7	**RANSAC**	Random Sample Consensus	Robust fitting method used to find the vanishing point.
8	**IPM**	Inverse Perspective Mapping	Warps the road into a top-down (bird's-eye) view.
9	**IRUVD**	Indian Road User Vehicle Dataset	Dataset used to train the vehicle detector.
10	**COCO**	Common Objects in Context	General detection dataset; pretrained model used only as fallback.
11	**GPU**	Graphics Processing Unit	Hardware used to speed up detection.
12	**TF32**	TensorFloat-32	GPU number format that speeds up matrix multiplication.
13	**ByteTrack**	–	Tracker that keeps one ID per vehicle across frames.
14	**Saturation flow**	Saturation flow rate	Maximum rate at which vehicles leave a queue during green.

CHAPTER 1

INTRODUCTION

Traffic at Indian intersections rarely behaves the way traffic engineering formulas assume. Lane markings are painted on the road, but drivers cross them, change lanes without warning, or squeeze a two-wheeler through a gap that was never a lane. When the signal turns green, the queue is often not arranged in lanes at all. Vehicles stand wherever they managed to reach first. This kind of indiscipline is generally expected to reduce the saturation flow rate, which is the highest rate at which vehicles can cross the stop line once the queue starts moving. Measuring saturation flow properly needs queue discharge headways over several signal cycles, and very little of this has been done from real video at Indian junctions. Doing it by hand takes too long, and sensors such as loop detectors and radar are too expensive to install at more than a handful of junctions.

LANE-GUARDIAN is our first step in that direction. It is a software-only pipeline that takes an ordinary recorded video from a fixed camera. It detects and classifies the vehicles, tracks them from frame to frame, estimates where the lanes are instead of relying on a surveyed lane plan, and flags specific lane discipline violations for review. At the end it summarises the lane-wise observed flow (vehicles and PCU per hour) with an indicative traffic-condition class for each lane, placed next to the violation statistics so that both can be read together for the same lane. No extra roadside sensing hardware is needed.

For vehicle detection we use a YOLOv10n [6] model trained in Phase I. YOLOv10n has an end-to-end design with no separate non-maximum-suppression step, which suits video processing. A detection by itself only says that a vehicle is present in one frame, whereas lane discipline depends on where the vehicle has been and where it is heading. So we link the detections into tracks with ByteTrack [8] and then check each track against the estimated lane geometry, frame by frame. The work was done in two phases. Phase I covered training the detector, the literature survey, and the detection and tracking parts (Objectives 1 and 2). Phase II added lane detection, the violation rules (Objective 3) and the traffic flow analysis (Objective 4). This report covers the whole project.

1.1 Problem Definition

Congestion at signalized intersections in Indian cities is made worse by dense traffic, a mixed fleet (cars, two-wheelers, autos and trucks sharing the same road width) and weak lane discipline. Monitoring today is either manual or sensor-based. Manual monitoring means a traffic constable directing vehicles and remembering who broke the rules, which does not scale and leaves no data behind. Sensors such as inductive loops are costly to retrofit, and they can show that a lane is performing badly but not why. There is no cheap, repeatable way to look at lane-wise traffic flow and lane discipline together from ordinary camera footage.

The project therefore has two parts. The first is a computer-vision pipeline that detects vehicles, tracks them and estimates lane geometry from recorded footage. The lane geometry is estimated automatically from road markings, perspective information and vehicle trajectories, and the operator can correct it by hand when the automatic result is unreliable. The second part uses that pipeline to pick out specific lane discipline violations as candidates for review. The system reports lane-wise traffic-flow figures alongside the violation statistics so that the two can be examined together, which makes the output more useful to a traffic engineer than a set of bounding boxes.

1.2 Objectives

- To collect and preprocess traffic intersection video datasets and carry out a literature survey on lane detection, vehicle tracking, and traffic flow analysis.
- To design and implement a computer vision-based system for lane detection, vehicle detection, vehicle classification, and multi-object tracking from recorded traffic videos.
- To develop a rule-based method for detecting lane discipline violations and computing lane-wise vehicle counts and movement trajectories, and to test it on recorded footage.
- To analyse lane-wise traffic flow (vehicle counts, PCU per hour and an indicative traffic-condition class) and to develop violation reporting, a dashboard, and automated Excel reports.

1.3 Scope of the Project Work

This is a fixed-camera project using a single (monocular) camera. We did not attempt multi-camera fusion or 3D reconstruction. The input is recorded video. Live CCTV or IP-camera integration is outside this report and is listed as future work. The trained detector uses five vehicle classes defined for this project: car, two-wheeler, auto-rickshaw, bus and truck. The flow module also holds PCU values for a few other class names (vans, tempos, tractors) in case a different model is loaded.

Lane geometry is estimated automatically from road markings, perspective information and vehicle trajectories. Many Indian roads have markings that are missing, faded or repainted, so manual correction is available: the operator can add, adjust or merge lanes by hand, and a calibration preview lets the lanes be checked before the analysis starts.

The violations used to demonstrate the system are Wrong-Way, Overspeeding, Lane Straddling, Unsafe Lane Change, Illegal Lane Change and Unauthorized Parking. These are rule-generated candidates, not confirmed offences. An operator can open each one next to its evidence image, approve it or leave it, and edit a suggested fine before the report is finalized. The fine values are configurable reference amounts for review. They are not a legal determination of the applicable penalty, and the software does not issue fines.

Speed and distance depend on the camera scale being inferred correctly, either from the spacing of dashed lane markings, from a reference speed supplied by the operator, or from an assumed focal length. Absolute speeds are therefore only indicative unless a real-world measurement is available, and the tool says so in its own output. Pedestrians and non-motorised traffic are not considered, and night or poor-light footage was not part of the test set. The flow figures are lane-wise observed flow: vehicle counts and PCU at a virtual counting line, scaled to an hourly rate, plus the peak 60-second observed flow. They are not a headway-based saturation flow over several signal cycles. The PCU weights are implementation-level assumptions used to compare traffic load between lanes. The Level of Service (LOS) in this report is a software-defined, indicative traffic-condition class and is not a Highway Capacity Manual LOS.

1.4 Organization of the Report

Chapter 1 Introduction:

Introduces the domain, the problem of lane indiscipline at Indian intersections, the objectives of LANE-GUARDIAN, and the scope of the work.

Chapter 2 Review of Literature:

Looks at existing work on vehicle detection, lane detection, and traffic violation monitoring, and lists the gaps this project tries to fill.

Chapter 3 System Requirement Specification:

Gives the functional and non-functional requirements, and the hardware and software needed to build and run the system.

Chapter 4 System Design:

Describes the architecture, the data flow diagrams, the use case and sequence diagrams, and the modules of the pipeline.

Chapter 5 Implementation:

Covers the tools used, the steps we followed, and the algorithms behind detection, tracking, calibration, and violation logic.

Chapter 6 Testing:

Describes the unit, integration, system, and functional testing done on the pipeline.

Chapter 7 Results and Discussion:

Goes through the output of a sample run: calibration, detection and tracking, the violation log, and the flow analysis, with screenshots of the modules.

Chapter 8 Conclusion and Future Scope:

Summarizes what we built and learned, and what we plan to do next.

CHAPTER 2

REVIEW OF LITERATURE

Before settling on YOLOv10n with ByteTrack, we read a number of recent papers on vehicle detection, lane detection, and traffic violation monitoring. Most of them pair a YOLO detector with some kind of multi-object tracker. We were not only trying to choose an architecture. We also wanted to see where these systems usually go wrong (lighting, faded lane markings, crowded traffic) so that LANE-GUARDIAN could be designed with those problems in mind. Related work includes CCTV-based violation detection for automated enforcement [1], violation detection on highways [2], two-wheeler rule violation detection with YOLO and OCR [12], a YOLOv8-based enforcement framework [13], and a study of traffic and violations at an Indian roundabout using UAV video [14]. Violations have also been found from vehicle trajectories [19], and [18] is a survey of deep learning for violation prediction. Other papers we read deal with lane and vehicle detection using YOLOv8 [10], lane detection with speed prediction [11], YOLOv8 speed estimation [16], multi-camera vehicle tracking [15], classification of large vehicles on narrow roads [21], YOLOv10 in foggy weather [5], and the Indian Road User Vehicle Dataset (IRUVD) [3].

2.1 System Study

2.1.1 Overview of Existing Work

Table 2.1: Literature Survey Table

Sl. No.	Title of the Paper and Year	Methodology and Datasets used	Merits	Limitations
1	Visual Detection of Traffic Incident through Automatic Monitoring of Vehicle Activities (2024) [20]	YOLOv8 for vehicle detection with DeepSORT for multi-object tracking on traffic video	High detection accuracy (~98%), real-time monitoring	No lane detection or lane-discipline violation rules

Sl. No.	Title of the Paper and Year	Methodology and Datasets used	Merits	Limitations
2	Visual Detection and Tracking of Lane Violations for Heavy Vehicles	HybridNets for lane detection, YOLOv8 for vehicle detection	95% lane detection mAP, speed estimation mAP of 93.1%	Does poorly in rain, fog, or low light
3	Multi-Object Vehicle Detection and Tracking Based on YOLOv8 and ByteTrack (2024)	YOLOv8 detection combined with ByteTrack tracking in traffic scenes	Tracking holds up in dense traffic	Needs a lot of GPU compute
4	Real-Time Vehicle Detection Using YOLOv8 Model	YOLOv8 deep learning detection applied to traffic videos	Fast detection, better localization of objects	Accuracy drops noticeably in dense traffic
5	Vehicle Speed Detection System Utilizing YOLOv8 (2024)	YOLOv8 detection with tracking and speed estimation from frame displacement	Speed estimated accurately without physical sensors	Needs careful camera calibration
6	Utilizing YOLOv8 for Enhanced Traffic Monitoring in ITS (2024) [4]	YOLOv8 applied to intelligent transportation system monitoring	Works well for real-time monitoring	Sensitive to lighting conditions
7	Vehicle Tracking Algorithm of Speed Estimation Based on Traffic Video Analysis (2024) [9]	YOLOv8 with OpenCV for vehicle detection and traffic density computation	Speed accuracy ≈95%, error ±2–3 km/h from CCTV video	Accuracy still falls in low-light CCTV conditions
8	Deep Learning-based Lane Detection for Intelligent Driving	CNN-based semantic segmentation of road video frames	Lane detection accuracy ≈94–96%, works on urban roads	Performance drops when markings are faded or blocked
9	Real-Time Traffic Flow Detection System Based on Video Image Processing	Vehicle detection and classification with rule-based lane violation logic	Classification accuracy ≈92–95%, detects lane misuse	Needs clear lane markings and high-resolution video
10	An Automated Classification of Vehicles and Violation Detection in Special Purpose Lanes	YOLO detection with rule-based violation logic	Vehicle detection accuracy ≈97%, real-time monitoring	Rules are made for special-purpose lanes, not general lane discipline

2.1.2 Research Gaps

Looking at the ten papers together, almost all use YOLOv8 or an older YOLO version, and almost none combine lane geometry, vehicle tracking and a violation rule engine in one pipeline. Papers 1, 3, 4, 5, 6 and 7 are good at detection and tracking but ignore lanes altogether. They can say how many vehicles crossed a line, but not whether those vehicles stayed in their lane. Papers 2, 8 and 9 are the other way round. Their lane detection is good, but vehicle behaviour and violation logic come second, and they report that accuracy drops in poor light or with faded markings, conditions that Indian roadside cameras can face. Paper 10 is the closest to what we need, since it classifies vehicles and detects violations by rules.

None of the papers presents lane-wise violation statistics next to lane-wise observed flow and a traffic-condition class, which is what a traffic planner actually works with. The surveyed work also does not specifically target the dense mix of two-wheelers, autos and cars on Indian roads, where markings are often faded and vehicles often sit across two lanes. LANE-GUARDIAN is aimed at three gaps: there is no single pipeline for vehicles, lanes and violations; existing work is weak on faded markings and poor lighting; and violation statistics are rarely reported together with lane-wise flow.

2.2 Proposed Work

LANE-GUARDIAN puts vehicle detection (YOLOv10n), multi-object tracking (ByteTrack [8]) and automatic lane estimation (vanishing-point estimation plus an inverse perspective, or bird's-eye, transform) into one pipeline, so that vehicle behaviour is checked against the lane boundaries frame by frame instead of in isolation. The lane estimate combines marking-based and trajectory-based evidence. When the markings are too faded to trust, the system can fall back on lanes inferred from vehicle trajectories, or the operator can add lanes by hand. A rule-based violation engine then flags lane straddling, unsafe and illegal lane changes, wrong-way driving, overspeeding and unauthorized parking against these lanes. Finally, a flow module turns the tracked counts into lane-wise observed flow, PCU per hour and an indicative traffic-condition (LOS) class, so violations and flow can be read side by side for each lane.

CHAPTER 3

SYSTEM REQUIREMENT SPECIFICATION

3.1 Functional Requirements

Functional requirements describe what the system should do: its functions, inputs, outputs, and how it deals with exceptions. They depend on the kind of software being built and on how the organization writing them works.

Table 3.1: Functional requirements

Requirement	**Description**
Video Ingestion	Let the user upload or select a recorded fixed-camera traffic video and read its basic details (resolution, duration, frame rate).
Automatic Camera Calibration	Estimate the vanishing point, lane boundaries, and an approximate metric scale of the road from painted markings, perspective information and vehicle trajectories.
Manual Calibration Fallback	Let the operator draw, correct or merge lane boundaries by hand when automatic calibration is unreliable (for example, badly faded markings).
Vehicle Detection and Classification	Detect vehicles in each frame using the trained YOLOv10n model and classify them as car, auto-rickshaw, two-wheeler, bus or truck.
Multi-Object Tracking	Assign each detected vehicle a track ID using ByteTrack and keep or re-associate it across frames, including short missed detections, so its path can be followed through the clip.
Lane Discipline Violation Detection	Check each vehicle's path against the estimated lane geometry and flag lane straddling, unsafe or illegal lane change, wrong-way driving, overspeeding, and unauthorized parking as rule-generated candidates.
Traffic Flow and Indicative LOS Computation	Convert vehicle counts to passenger car units (PCU), work out lane-wise flow, PCU/hour and observed peak 60-second flow, and give each lane an indicative LOS (traffic-condition) class.
Report and Dashboard Generation	Produce an annotated output video, a multi-sheet Excel report (Summary, Violations, Lane Analysis, Vehicle Classes, Tracks, Lane Changes, Calibration), and an HTML dashboard.
Violation Review and Fine Approval	Let an operator go through each logged violation next to its evidence image, mark it approved or not, and edit a configurable suggested fine value, then save an updated Excel report that records those decisions.

3.2 Non-Functional Requirements

Non-functional requirements are not tied to one function of the system. They cover things like reliability, response time, and storage use, or they put limits on input/output devices and data formats.

Table 3.2: Non Functional Requirements

Requirement	**Description**
Performance	GPU-assisted processing of recorded traffic videos.
Usability	Operator-oriented Gradio interface that guides the user through upload, calibration, analysis and review.
Maintainability	Modular Python implementation with separate stages for calibration, detection and tracking, violation rules, and reporting.
Reliability	The pipeline should execute consistently on the supported recorded-video inputs and report warnings when calibration is unreliable.
Compatibility	Runs on a local workstation (Windows or Linux) with a browser-based interface.

3.3 Hardware Requirements

The hardware requirements give the minimum and recommended setup needed to develop, train, and run the application. Since the project trains a deep learning model and runs it on recorded video through a web interface, the machine needs decent processing power, memory, and storage.

Table 3.3: Hardware Requirement Specification

Component	**Specification**
Processor	Intel Core i5 (12th Gen) or AMD Ryzen 5 (5800H / 7000 series) or better, for multi-threaded frame preprocessing and tracking.
Graphics (GPU)	NVIDIA RTX A2000 (12GB GDDR6) GPU, compute capability of 8.6.
Memory (RAM)	16 GB minimum, 32 GB recommended, so that batched frames and calibration buffers do not cause heavy swapping.
Storage	512 GB SSD, for fast reading and writing of video, model weights, evidence images and logs.
Camera / Video Source	Recorded video from a fixed-angle traffic camera, ideally 1080p. Lower-resolution footage can also be processed.

3.4 Software Requirements

The software requirements list the tools, frameworks and libraries used to build and run the application. They cover the operating environment, the programming language, the detection and tracking framework, the image and video processing libraries, the reporting libraries and the user interface.

Table 3.4: Software Requirement Specification

Component	**Specification**
Operating System	A local Windows or Linux machine with Python can be used for development and testing.
Programming Language	Python 3.10, with supporting libraries TensorFlow 2.10.0, Keras 2.10.0 ,Ultralytics 8.4 and NumPy 1.23.5.
Detection and Tracking	Ultralytics framework with PyTorch: YOLOv10n for vehicle detection (custom `best.pt` weights, COCO YOLOv10n as fallback) and ByteTrack for multi-object tracking.
Image and Video Processing	OpenCV for marking extraction , the Hough transform, perspective transforms and phase correlation; NumPy for numerical work; Pillow for images; FFmpeg for video conversion.
Data and Reporting	pandas for tables and openpyxl for the multi-sheet Excel report; an HTML dashboard for the run summary.
User Interface	Gradio browser-based interface for video upload, calibration, analysis, and the Review & Fines step.
Development Environment	VSCode on a local workstation.

CHAPTER 4

SYSTEM DESIGN

4.1 Design Overview

LANE-GUARDIAN is a sequential computer-vision pipeline with five stages: input video, lane geometry estimation, vehicle detection and tracking, rule-based violation detection, and traffic-flow reporting. A clip comes in, and the system solves for the camera geometry and lane layout, tracks the vehicles through the clip, checks each vehicle's position against the detected lanes, and produces traffic-flow summaries with an indicative Level of Service (LOS) estimate. We keep lane geometry separate from the violation rules on purpose, because a poor lane estimate and a badly tuned rule give output videos that look identical. The calibration step therefore writes diagnostic images and warnings and reports a geometry-consistency score, so an inaccurate lane position can be spotted and corrected, automatically or by hand, before it reaches the violation stage. The final system combines temporal stabilization of the lane geometry, camera-motion tracking and robust manual lane merging, and the HTML dashboard and the Review & Fines page help the operator check the results.

We built it this way because in traffic footage the road geometry usually fails before the rules do, so each stage hands the next one something a person can look at and verify. LANE-GUARDIAN never issues a fine on its own: every flagged violation goes to the Review & Fines page with the frame, the lane overlay and the vehicle's track, so the operator makes the final call. The LOS value is labelled indicative on purpose, since it comes from a single camera view and short clips and suits comparing time windows, not formal traffic-engineering reports. Next we plan to test it on more camera angles, night and rain footage, and slow, congested traffic, where lane markings and vehicle tracks are hardest to read.

4.2 System Architecture

The system architecture shows how the parts of the system fit together. LANE-GUARDIAN is organised as a sequential pipeline of five stages, where the output of each stage is the input of the next.

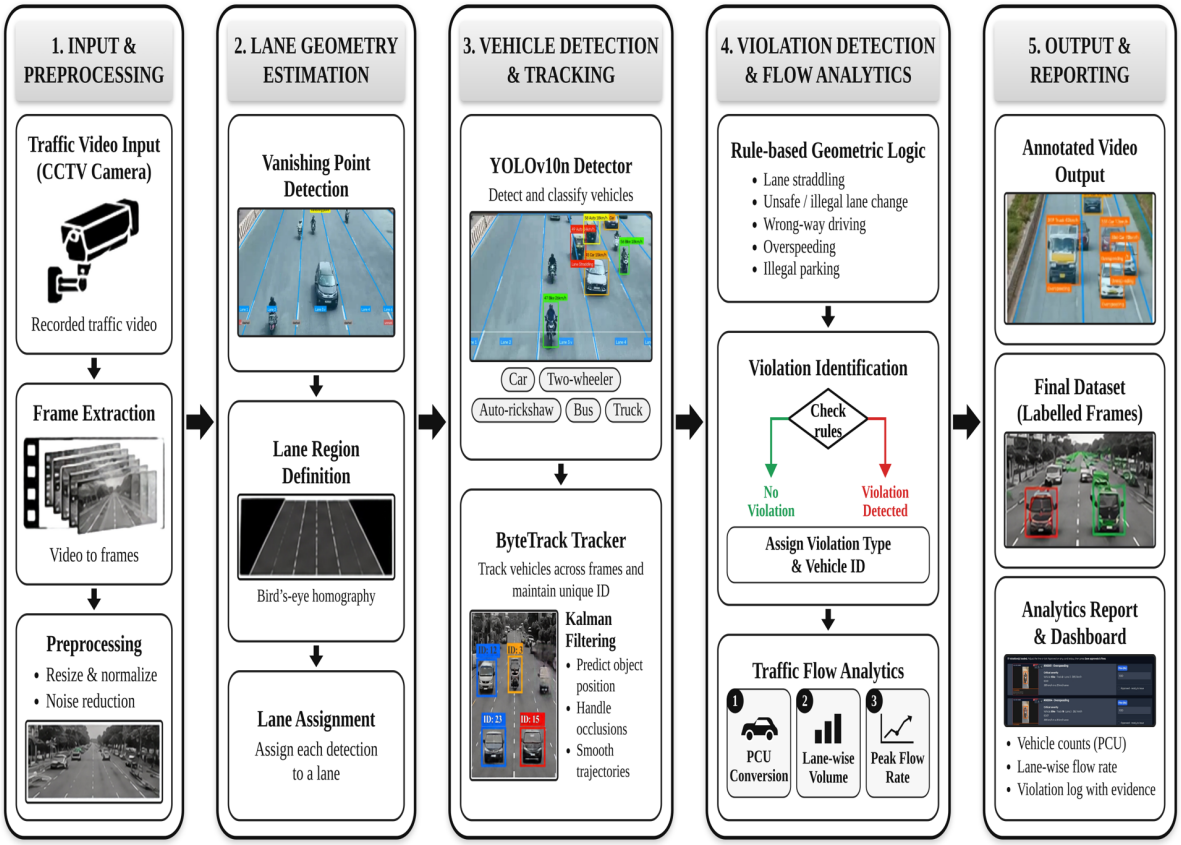


Figure 4.1: System Architecture

The system architecture is shown in Figure 4.1. The complete working process of the Lane-Guardian system starts from capturing the traffic video and ends with generating the analyzed output.It has five stages:

1. **Input & Preprocessing:** Reads the recorded traffic video, samples frames for calibration, gets details such as resolution and frame rate, and estimates a background image and an activity mask that separate static road paint from moving vehicles.

2. **Lane Geometry & Calibration:** Uses OpenCV-based marking extraction (paint masking with top-hat filtering, Probabilistic Hough segments), RANSAC vanishing-point estimation, trajectory-based cues, a bird's-eye perspective transform and lane fitting to estimate the lane

boundaries, with temporal stabilization and manual lane correction when automatic calibration is unreliable.

3. **Vehicle Detection & Tracking:** Runs the trained YOLOv10n model to identify vehicle classes (cars, two-wheelers, autos, trucks, buses) frame by frame, and uses ByteTrack for multi-object tracking and vehicle ID assignment, recording each vehicle's path across the frame.

4. **Violation Detection:** A rule-based stage that compares each vehicle's path with the estimated lane polygons to flag lane straddling, unsafe or illegal lane changes, wrong-way driving, overspeeding, and unauthorized parking as rule-generated candidates.

5. **Traffic Analytics & Reporting:** Counts vehicles in each lane at a virtual counting line, converts them to PCU flow per hour, reports the observed peak 60-second flow and an indicative LOS class from mean speed, reports the share of tracked vehicles with at least one rule-generated violation, and produces the annotated video, Excel report, HTML dashboard and the Review & Fines output.

4.3 Data Flow Diagrams

4.3.1 Data Flow Diagram - Level 0

DFD-0 is shown in Figure 4.2. It treats the whole system as one process: a recorded video source goes in and a Traffic Analysis Report (annotated video, Excel workbook, dashboard) comes out. It is a context diagram, so none of the inner stages are shown.

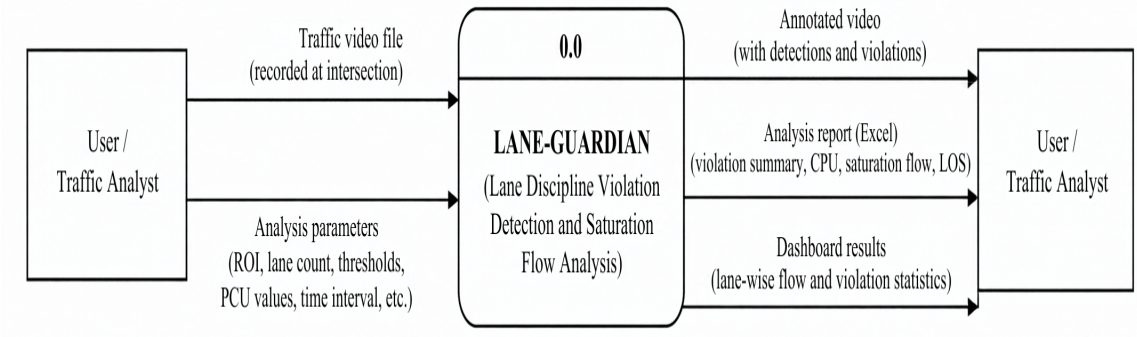


Figure 4.2: Data Flow Diagram - Level 0

4.3.2 Data Flow Diagram - Level 1

DFD-1 is shown in Figure 4.3. It splits the system into four main processes: Preprocessing (frame sampling, background estimation, calibration), Detection and Tracking (YOLOv10n inference and ByteTrack association), Violation Analysis (lane-geometry checks and rule evaluation), and Traffic Analytics (PCU conversion, lane-wise flow, observed peak flow and indicative LOS classification). The data stores at this level are the model weights (`best.pt`), the calibration file, and the event log of violations.

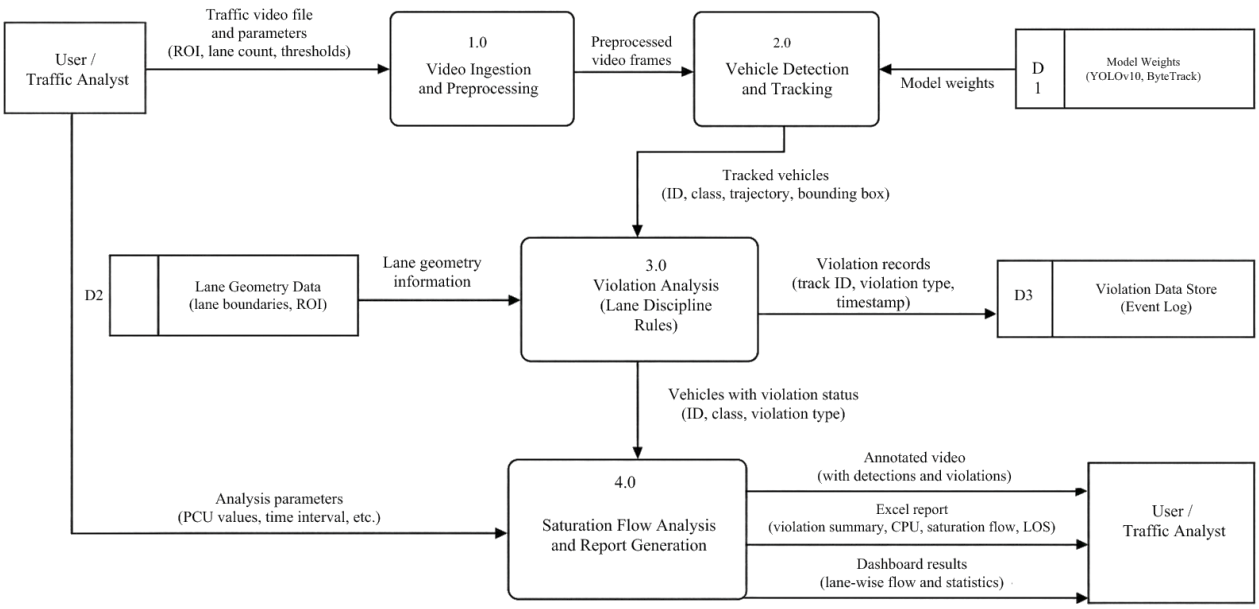


Figure 4.3: Data Flow Diagram - Level 1

4.3.3 Data Flow Diagram - Level 2

DFD-2 is shown in Figure 4.4. It opens up the Violation Analysis process from Level 1. It shows the point-in-polygon check that decides whether a vehicle's ground point (the bottom centre of its bounding box) or its body width has crossed a lane boundary, and how that check feeds the state machine for each violation type (for example, how long a vehicle has to stay over a line before it counts as straddling and not just a short wobble).

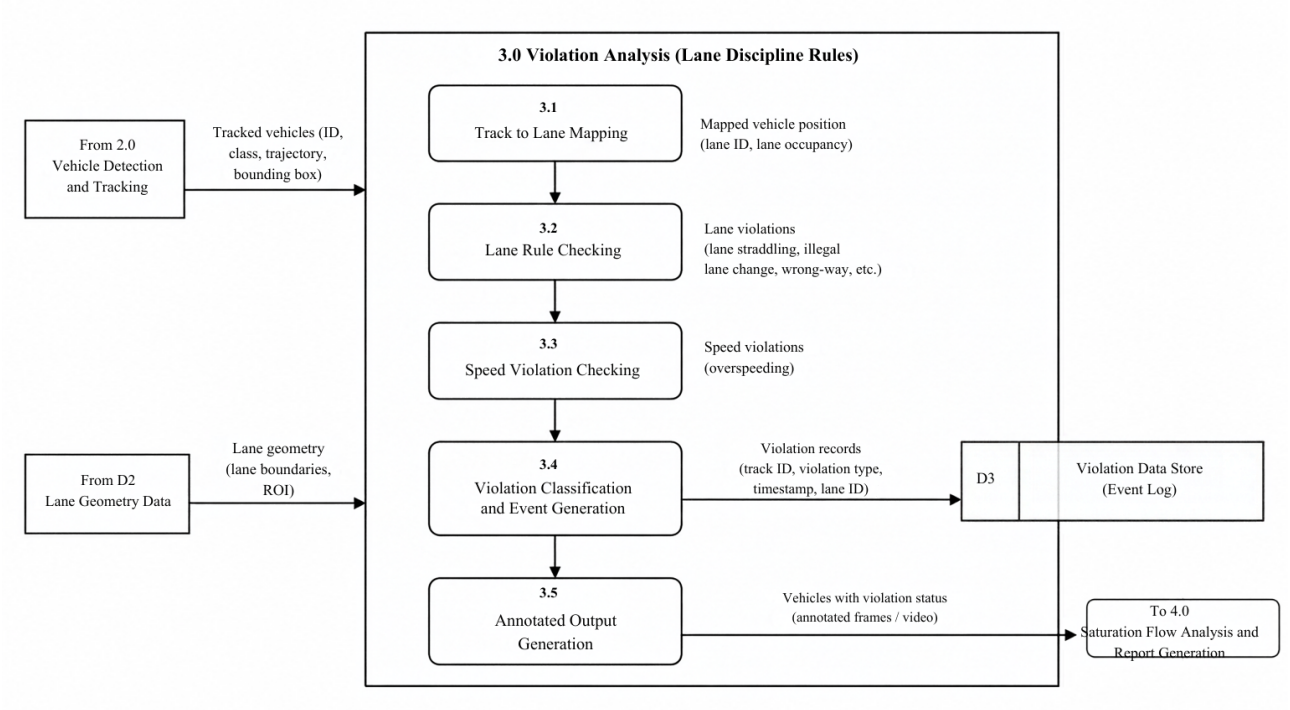


Figure 4.4: Data Flow Diagram - Level 2

4.4 Use Case Diagram

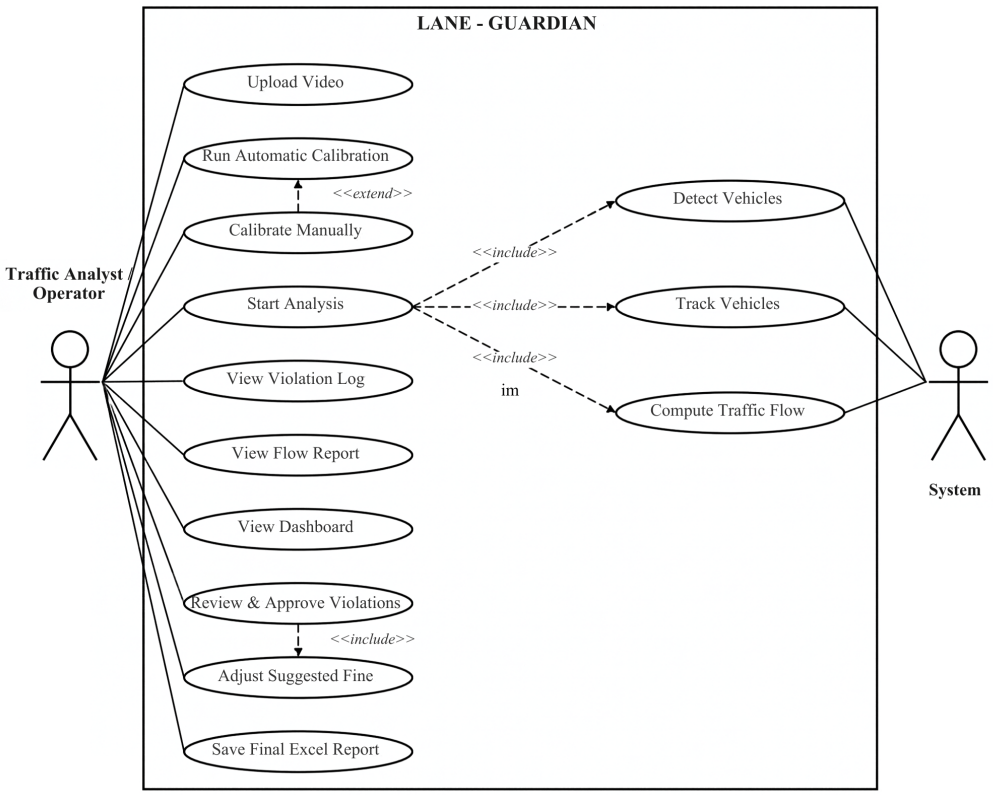


Figure 4.5: Use Case Diagram

The use case diagram is shown in figure 4.5. The external actor is the Traffic Analyst/Operator. The system itself is the system boundary and not an actor. Inside the boundary, the operator can upload a video, calibrate the lanes (automatically, with manual correction when needed), run the analysis, view the violations, view the traffic analytics, review the violation evidence, edit or approve the suggested fine, and export the report. Detection, tracking and flow computation are internal processing steps that run once the operator selects “Start Analysis”.

4.5 Sequence Diagram

The sequence diagram is shown in figure 4.6. It follows one full analysis run. The operator uploads a video, and the system samples frames and calibrates. It then sends back a geometry-consistency score and a diagnostics image for the operator to check. The operator confirms the lanes or adjusts them by hand, and the system runs the detector and tracker over the frames. The rule engine looks for violation candidates as the tracks update. After the last frame, the flow analysis and reporting modules run and hand the Excel report and dashboard to the operator. The operator can then open the Review & Fines step, go through each violation against its evidence image, approve it and edit the suggested fine, and save an updated Excel report with those decisions in it.

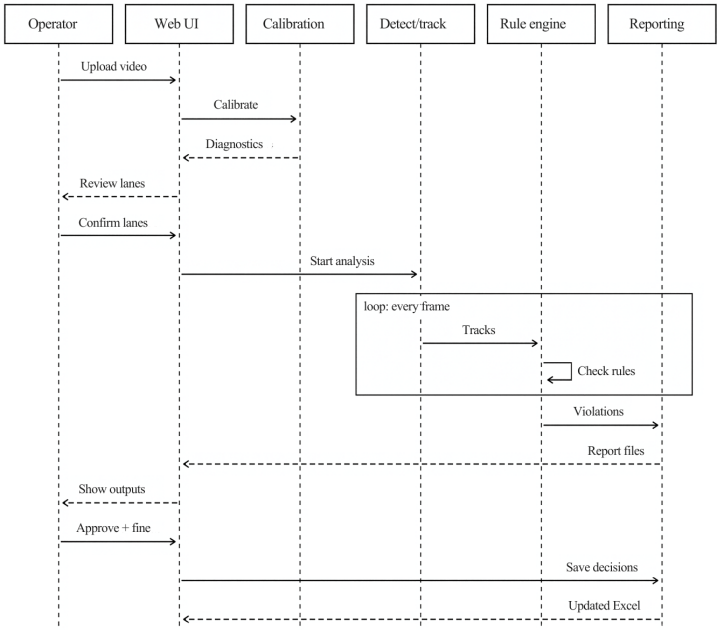


Figure 4.6: Sequence Diagram

4.6 Modules of the Project

The modules are

1. Module 1: Video Preprocessing and Camera Calibration
2. Module 2: Vehicle Detection and Classification
3. Module 3: Multi-Object Tracking
4. Module 4: Lane Discipline Violation Engine
5. Module 5: Traffic Flow and Observed-Flow Analysis
6. Module 6: Reporting, Review, Dashboard, and Web Interface

4.6.1 Module 1: Video Preprocessing and Camera Calibration

Functionality: Samples frames across the clip and estimates a background and an activity mask, which separate static lane paint from moving vehicles. It then detects lane-marking pixels, estimates the vanishing point with a RANSAC fit over marking segments (with a trajectory-based vanishing point as a cross-check), builds an inverse-perspective (bird's-eye) warp, and fits lane boundary curves in the warped view. The geometry is stabilized over time, small camera motion is tracked, and the operator can add, correct or merge lanes manually.

Input: Recorded traffic video file.

Output: Calibration file (vanishing point, boundary curves, metric scale, consistency score), diagnostics image and warnings.

Algorithm used: Top-hat marking extraction with the Hough transform, RANSAC vanishing-point estimation, inverse perspective mapping (IPM), temporal boundary consensus, curve fitting and dash-period scale calibration (Algorithms 3 and 4, Chapter 5).

4.6.2 Module 2: Vehicle Detection and Classification

Functionality: Runs the trained YOLOv10n detector on the analysed frames to find vehicles and give each one a class label (car, auto-rickshaw, two-wheeler, bus or truck).

Input: Video frames and the custom YOLOv10n weights (`best.pt`), with COCO YOLOv10n as fallback.

Output: Boxes with class and confidence for each frame.

Algorithm used: YOLOv10n (NMS-free, single-stage object detector).

4.6.3 Module 3: Multi-Object Tracking

Functionality: Links per-frame detections into vehicle tracks. ByteTrack keeps or re-associates vehicle IDs across frames, including short missed detections or occlusion, and the full path of each track is stored.

Input: Per-frame detections from Module 2.

Output: Vehicle tracks with ID, class and time-stamped path.

Algorithm used: ByteTrack via the Ultralytics tracking interface, with project-specific thresholds and track buffer.

4.6.4 Module 4: Lane Discipline Violation Engine

Functionality: Projects each track's path onto the lane geometry and checks it against the rules: how far and how long the vehicle is over a boundary line, abrupt lane changes, movement against the lane's flow direction, speeding, and standing in a travel lane longer than the set parking duration. Each rule-generated candidate is recorded with a severity.

Input: Vehicle tracks (Module 3), lane geometry (Module 1).

Output: Violation log (type, severity, confidence, lane, time) with an evidence crop.

Algorithm used: Point-in-polygon lane membership checks and rule-based checks with confidence and time thresholds for each violation.

4.6.5 Module 5: Traffic Flow and Observed-Flow Analysis

Functionality: Counts each tracked vehicle once, when it crosses a virtual counting line, and notes the lane it was in. The counts are converted to PCU and scaled to one hour to give lane-wise flow in vehicles/hour and PCU/hour, and the observed peak 60-second flow is reported for each lane. Each lane also gets an indicative LOS class (mean speed against the speed limit) and an occupancy measure.

The share of tracked vehicles with at least one violation candidate is shown next to the flow for each lane. It is descriptive only and is not subtracted from the flow.

Input: Violation log (Module 4), classified vehicle tracks.

Output: Lane-wise flow, PCU/hour, observed peak flow, occupancy, indicative LOS class and violation share.

Algorithm used: PCU-weighted counts at a counting line, and LOS bands based on the ratio of mean speed to the configured speed limit. The PCU weights are implementation-level assumptions for comparing traffic load and should be replaced with a standard's values for formal studies.

4.6.6 Module 6: Reporting, Review, Dashboard, and Web Interface

Functionality: Produces the final outputs: an annotated video; a multi-sheet Excel workbook (Summary, Violations, Lane Analysis, Vehicle Classes, Tracks, Lane Changes, Calibration); an HTML dashboard; and a Gradio web interface for running the pipeline without code. It also holds the Review & Fines step, where each logged violation is shown next to its evidence image so the operator can approve it and edit a suggested fine. Saving writes the approvals and fines into a fresh copy of the Excel report, so the report records the operator's decision, not just the rule output.

Input: Outputs of Modules 1–5 and the operator's approve/fine decisions.

Output: Annotated video, Excel report (with approval status and fines), HTML dashboard, web interface.

Algorithm used: pandas/openpyxl report generation, evidence-crop extraction and a configurable suggested-fine lookup.

CHAPTER 5

IMPLEMENTATION

Implementation is the step where the design becomes a working system. It decides whether the project succeeds, so it needs to be planned and controlled with care.

5.1 Steps for Implementation

In Phase I we turned the problem statement into a working pipeline that reads intersection footage, classifies the different vehicle types and keeps one tracking ID for each vehicle from the moment it enters the frame until it leaves. The detector was trained on Indian traffic images and reached a mAP@50 of 87.68%, with class-wise accuracy of 90.4% for cars and 89.5% for auto-rickshaws. ByteTrack kept the IDs stable from frame to frame, including during partial occlusion and sudden changes in speed. A live demonstration stream also showed that the same pipeline can work on continuous input and not only on saved video. In Phase II we added lane calibration, the violation rules, the flow analysis and the Review & Fines step on top of this base. The sections below cover how the system was set up, the data it was run on, how it was debugged, and the steps we followed to build it.

5.1.1 Installation of Hardware and Software Utilities

All experiments were run on a dedicated workstation with an NVIDIA RTX A2000 (12GB GDDR6) GPU, compute capability 8.6. TensorFloat-32 (TF32) precision was enabled so that matrix multiplications use the Tensor Cores of the GPU. The software was developed in Python 3.10, with TensorFlow 2.10.0, Keras 2.10.0 and NumPy 1.23.5 for GPU-accelerated computation. At the start of each session we installed Ultralytics (YOLOv10n), OpenCV, Gradio, pandas and openpyxl, along with `ffmpeg` for video conversion. The trained detector weights (`best.pt`) are kept in the working folder and loaded from there when the tool starts.

5.1.2 Sample Data Used

Training data: The detector is the YOLOv10n model we trained in Phase I on the Indian Road User Vehicle Dataset (IRUVD) [3], which has over one thousand annotated images of Indian road traffic in five classes (car, two-wheeler, auto-rickshaw, bus, truck). We load the detector from `best.pt` instead of the generic COCO weights, because COCO does not separate autos from cars and Indian traffic analysis needs that. If the checkpoint is missing, the code falls back to COCO YOLOv10n and drops the non-vehicle classes.

Test videos: For testing we used short recorded clips of 30 to 60 seconds from fixed-angle cameras. They came from the Electronic City Skywalk and other roads in Electronic City and near Nice Road, along with footage of a highway road in China and a road video from the IRUVD dataset. The main clip used in Chapter 7 is 99 seconds of mixed intersection traffic. We deliberately picked clips with partly faded lane markings, so that manual lane correction and the trajectory-assisted calibration were exercised and not only the easy case.

5.1.3 Debugging Phase

Debugging the calibration pipeline took many rounds. The tool went through many versions (the current one is V30 PRO). The main problems and their fixes were:

- The body of a stationary vehicle was mistaken for lane paint and pulled the lane lines off course. We now estimate a background image and an activity mask from the sampled frames.
- At some camera angles the vanishing point falls outside the frame. In that case the trajectory-based estimate or a default point above the road is used, and the operator is warned.
- A boundary seen in only one frame could change the lane count. A boundary is now accepted only if it appears in enough of the sampled frames.
- Small camera movement made the lane overlay drift. The movement is now tracked against the calibration frame.
- Wrong-way vehicles were missed in lanes with little traffic. We lowered the number of tracks needed before a lane's direction of travel is trusted.

- Detections were lost when the class name from the model did not match the project's class names. The class-name matching was widened.
- Violation boxes flashed for a single frame and were easy to miss. They now stay visible on the video for a short time.

Each fix was first checked on synthetic test cases and then on real footage, because re-running the full video analysis after every small change was too slow.

5.1.4 Implementation Steps

1. **Vehicle detection.** Detection runs on every second frame at an inference size of 640 pixels with a confidence threshold of 0.18, using the YOLOv10n weights from `best.pt`.
2. **Tracking.** ByteTrack is used through the Ultralytics tracking interface with these settings: high-confidence threshold 0.40, low-confidence threshold 0.10, new-track threshold 0.60, matching threshold 0.90, score fusion on, and a track buffer of 120 frames so that tracks survive short missed detections or occlusions.
3. **Lane calibration.** A background image and an activity mask are estimated from sampled frames. Paint pixels are extracted with a top-hat filter in the LAB colour space, line segments are found with the Probabilistic Hough Transform, and the vanishing point is estimated with RANSAC and cross-checked against vehicle trajectories. A bird's-eye transform is then used to fit the lane boundaries. If the markings are unreliable, lanes can be inferred from vehicle trajectories, and lanes added by the operator are merged with the automatic ones.
4. **Violation detection.** Each tracked vehicle is checked against the lane model for lane straddling, unsafe and illegal lane changes, wrong-way driving, overspeeding and unauthorized parking. The main thresholds can be changed in the Analyse tab of the interface.
5. **Flow analysis.** Vehicles are counted once at a virtual line, converted to PCU, and scaled to lane-wise hourly flow, with an indicative Level of Service from the mean speed of each lane.
6. **Reports and interface.** The annotated video, the Excel report and the HTML dashboard are produced from the same run, and the whole pipeline is run through a Gradio interface.

7. **Review & Fines.** We included this step because a raw violation log cannot be acted on by itself: someone still has to look at each flagged event and decide whether it is a real violation. Each violation opens as a card showing the evidence image, vehicle class, lane, speed and a suggested fine. The default values are configurable reference amounts (for example Rs. 500 for lane straddling and Rs. 1000 for wrong-way driving, with a higher default of Rs. 2000 for trucks and buses on overspeeding) and every figure can be edited. They are not a legal determination of the applicable penalty. Approving a card and saving writes the decision into a fresh copy of the Excel report.

5.2 Implementation Issues

Implementation turns the design into source code. The main aim is to write code and internal comments so that it is easy to check the code against the specification, and easy to debug, test, and change. We did this by keeping the code as clear and simple as we could in every module.

The goals of implementation are as follows.

- Minimize the memory required.
- Maximize output readability.
- Maximize source text readability.
- Minimize the number of source statements.
- Minimize development time.

One problem specific to this project is that geometry bugs and detection bugs look the same in the output video, since both appear as a box or a line in the wrong place. Keeping the calibration diagnostics image and warnings as a separate output, instead of folding calibration silently into the main run, makes debugging easier and lets the operator confirm the lane geometry before spending time on the full video.

5.3 Algorithms

The six main algorithms of LANE-GUARDIAN are described here, each as the sequence of steps it follows. All numbers (thresholds, sizes, durations) are the values used in the implementation.

1. **Algorithm 1: YOLOv10n Vehicle Detection**

   Finds and classifies the vehicles in each analysed frame using the custom-trained YOLOv10n model [6]. **Input:** video frames and `best.pt`. **Output:** vehicle boxes with class and confidence.

   Step 1: Load `best.pt` on the GPU (CPU if no GPU is present). If it is missing, load the COCO `yolov10n.pt` instead.

   Step 2: Read the video and keep every second frame (frame stride 2), resized to at most 1280 pixels wide.

   Step 3: Run the detector at inference size 640 with a confidence threshold of 0.18. Its one-to-one head gives the final boxes directly, so no NMS step is needed.

   Step 4: Apply class filtering: drop classes that are not vehicles (person, traffic light and similar) and keep only the vehicle categories, such as cars, buses, trucks, auto-rickshaws and two-wheelers.

   Step 5: Pass the boxes, classes and confidences to the tracker (Algorithm 2).

2. **Algorithm 2: ByteTrack Multi-Object Tracking**

   Gives each vehicle an ID that stays the same across frames, using a two-stage match so that a briefly hidden vehicle is not dropped [8]. **Input:** detections from Algorithm 1. **Output:** tracks with ID, class and path.

   Step 1: Set the tracker thresholds: high 0.40, low 0.10, new-track 0.60, matching 0.90, and a track buffer of 120 frames.

   Step 2: Predict the next position of every track with the Kalman filter.

   Step 3: First match: high-score detections (0.40 or more) to the tracks by IoU.

   Step 4: Second match: still-unmatched tracks to low-score detections (0.10 to 0.40), so a partly hidden vehicle keeps its ID.

Step 5: Start a new track only from an unmatched detection of 0.60 or more. Keep an unmatched track as lost for up to 120 frames, then remove it.

Step 6: Store the ground point of each track at every frame and fix its class by majority vote over the track.

3. **Algorithm 3: RANSAC Vanishing-Point Estimation**

Finds the point where the parallel road lines meet in the image, from the markings and from vehicle paths. **Input:** marking segments (top-hat paint extraction and Hough transform) and prescan tracks. **Output:** the vanishing point and a confidence value.

Step 1: Turn each marking segment into a line and weight it by its length.

Step 2: Repeat 600 times: pick two non-parallel segments, intersect them, and score the point by the weight of the lines passing within 1% of the frame diagonal of it.

Step 3: Accept the best point only if at least 12% of the segments (and no fewer than three) support it, then refine it by weighted least squares.

Step 4: Do the same on long vehicle paths (RANSAC, 500 iterations) to get a trajectory vanishing point.

Step 5: If the marking point has confidence of 0.45 or more, use it and keep the trajectory point only as a cross-check.

Step 6: Otherwise use the trajectory point, or the weighted average if the two are close.

Step 7: If neither is found, use a default point above the road and warn the operator.

4. **Algorithm 4: Bird's-Eye Transform and Lane-Boundary Estimation**

Warps the road to a top view and finds the lane boundaries there. **Input:** marking mask, road corridor, vanishing point, prescan tracks. **Output:** lane polygons, boundary types, lane directions and the geometry-consistency score.

Step 1: Build the inverse perspective mapping from the road corridor and vanishing point, warping it to a 640×900 bird's-eye image.

Step 2: Warp the marking mask, add up the marking pixels in each column to get a profile, and estimate the lane spacing from it.

Step 3: Find the boundaries by temporal consensus: over up to 48 frames, cluster the

marking positions and keep a boundary only if it appears in at least 28% of the frames (and at least 8). Use a regular lane grid as a fallback.

Step 4: Fit each boundary as a curve with one shared curvature, merge boundaries closer than half a lane and reject those with support below 0.18. If fewer than two remain, ask the operator to draw the lanes.

Step 5: Map the curves back to the image, label each boundary as solid, dashed or unmarked, and build one polygon per lane.

Step 6: Set each lane's direction from the vehicles in it and compute the geometry-consistency score (0.45 boundary support + 0.35 vanishing-point confidence + 0.20 parallelism). Warn the operator if it is below 0.35.

5. **Algorithm 5: Rule-Based Lane Violation Detection**

Checks every tracked vehicle against the lane model at each analysed frame and logs a violation candidate when a rule is met. The values below are defaults, and the main ones can be changed in the Analyse tab. **Input:** tracks, lane model, metric calibration. **Output:** the violation log with evidence crops and suggested fines.

Step 1: Find the lane polygon that contains the ground point of the vehicle, and update its metric position and speed.

Step 2: Lane change: once the vehicle has stayed 0.55 s in a new lane, log *Illegal Lane Change* if a solid boundary was crossed, or *Unsafe Lane Change* if it crossed two or more lanes or took under 0.85 s.

Step 3: *Lane Straddling*: overlap beyond lane edge above 0.28 of the class width for 1.8 s, It will be flagged as lane straddling.

Step 4: *Wrong-Way*: movement opposite to the lane direction for at least 1.0 s and 3 m (once per vehicle).

Step 5: *Overspeeding*: above the speed limit (80 km/h default) for at least 0.4 s. *Unauthorized Parking*: moved less than 1.2 m over 30 s (once per vehicle).

Step 6: Suppress repeats with a cooldown of 6 to 15 s, save an evidence crop, and attach a suggested fine (Rs. 500 to Rs. 2000), marked as not approved, for the Review & Fines tab.

6. **Algorithm 6: Observed Flow and Indicative LOS**

   Counts vehicles at a virtual line and converts the counts into lane-wise flow, PCU flow, peak flow and an indicative traffic-condition class. **Input:** tracks and lane model. **Output:** the lane analysis table.

   Step 1: Place a counting line at 72% of the frame height. Count each vehicle once, in its lane, when its ground point first crosses it.

   Step 2: Weight each counted vehicle by its PCU value: car 1.0, auto-rickshaw 0.8, two-wheeler 0.5, bus and truck 3.0.

   Step 3: Scale to one hour: count $\times$ 3600 divided by the clip duration (vehicles/hour and PCU/hour). The observed peak 60-second flow is the largest count in any 60-second window, times 60.

   Step 4: Divide each lane's mean speed by the speed limit and set the indicative LOS: A for 0.90 or more, B 0.75, C 0.60, D 0.45, E 0.30, and F below that. This is a software-defined class, not a Highway Capacity Manual LOS.

   Step 5: Compute lane occupancy (vehicle observations over analysed frames, as a percentage) and the violation share (vehicles with at least one candidate over stable tracked vehicles).

CHAPTER 6

TESTING

Testing this pipeline is not just a matter of checking that the code runs. Most of the failures we cared about are geometric or statistical, not syntax errors. A lane boundary drawn well off the real marking, or a violation logged with the wrong severity, throws no error and is simply wrong. Quantifying such errors needs manually annotated ground truth. The tests below therefore check that individual functions behave as specified and that the stages work together, and the end-to-end checks are qualitative.

6.1 Test Environment

Testing was done on the same workstation (NVIDIA RTX A2000 GPU) used for development. Single functions (PCU lookup, LOS classification, suggested-fine lookup) were extracted from the notebook and run on chosen inputs. Recorded intersection footage was used to check that the stages work together and produce their outputs. We also made manual spot checks on selected output frames and violation evidence. These checks are not a ground-truth accuracy evaluation.

Additional testing focused on checking the output of each module, including lane boundaries, vehicle detection, tracking IDs and violation evidence. The generated annotated video, Excel report and HTML dashboard were also inspected to check their basic completeness and consistency.

The interaction between modules was examined to ensure that vehicle detections, tracks and violation records were passed to the subsequent stages. However, since manually annotated ground-truth data was not available, quantitative accuracy measurements could not be established. The testing therefore focuses on functional behaviour and qualitative verification of the system.

The system was also tested under different traffic conditions to observe how it handled vehicle movement and violation detection. The generated outputs were checked for readability and proper organisation.

6.2 Unit Testing

Table 6.1: Unit Testing

Steps	Test Data	Expected Results	Observed Results	Remarks
Step 1	PCU lookup called with the class names car, auto-rickshaw, bus, truck and motorcycle	Weights 1.0, 0.8, 3.0, 3.0 and 0.5 respectively	Function returned 1.0, 0.8, 3.0, 3.0 and 0.5	Pass
Step 2	LOS function called with mean speeds equal to 0.95, 0.80, 0.65, 0.50, 0.35 and 0.10 of an 80 km/h limit	Classes A, B, C, D, E and F respectively	Function returned A, B, C, D, E and F	Pass
Step 3	Suggested-fine lookup for overspeeding by a car and by a truck, and for lane straddling by a car	Default values of Rs. 1000, Rs. 2000 and Rs. 500	Function returned 1000, 2000 and 500	Pass
Step 4	Detector validation split (IRUVD)	Detector metrics recorded for the `best.pt` checkpoint	mAP@0.5 of 87.4% overall, 90.4% for cars and 89.5% for auto-rickshaws on the validation split (not an independent test set)	Recorded

The unit tests ran single functions, such as the PCU lookup, LOS classification and suggested-fine lookup, on chosen inputs and compared the output with the expected value. All of them passed. They show that these functions behave as written, not how accurate detection or violation flagging is.

6.3 Integration Testing

Table 6.2: Integration Testing

Steps	Test Data	Expected Results	Observed Results	Remarks
Step 1	Short recorded test video, calibration module only	Calibration file and diagnostics image produced	Calibration file and diagnostics image were produced (see the calibration preview in Chapter 7)	Pass
Step 2	Calibration output fed into detection module	Detector loads and runs on the analysed frames using the calibrated frame size	Detector ran without geometry mismatch errors	Pass
Step 3	Detection output fed into tracking module	Tracks start and update from frame to frame	Track IDs were assigned and paths recorded; no ground-truth ID evaluation was done	Functional pass

Steps	Test Data	Expected Results	Observed Results	Remarks
Step 4	Tracking output fed into violation engine	Rule-generated violation candidates logged with lane, time and evidence image	Violation candidates and evidence images were produced; a selected few were inspected manually, without a ground-truth comparison	Functional pass
Step 5	Violation log fed into flow-analysis module	Lane-wise count, PCU, flow, peak flow and LOS computed for every calibrated lane	Lane analysis table produced with an entry for each calibrated lane	Functional pass
Step 6	Full pipeline output fed into Excel report builder	Excel workbook generated with all seven expected sheets	Workbook generated with Summary, Violations, Lane Analysis, Vehicle Classes, Tracks, Lane Changes, and Calibration sheets	Pass
Step 7	Full pipeline output fed into dashboard builder	HTML dashboard shows summary cards and tables	Dashboard was generated and displayed the summary cards and tables	Pass
Step 8	Gradio interface end to end: upload, calibrate, analyze, download	Interface shows the progress of each stage and gives downloadable outputs	Interface completed the stages and outputs could be downloaded	Pass
Step 9	Manual lane addition merged with automatic calibration	Manually added lanes kept alongside the automatically estimated ones in one lane model	Merged lane overlay was displayed (see Chapter 7); the final lane count depends on the clip and is reported with the run	Functional pass
Step 10	Violation log fed into the Review & Fines step, one card approved with its fine amount changed	Approval and the edited fine are saved and show up in a re-exported Excel report	Updated report recorded the approval status and the edited fine	Pass

Each step passes one module's output to the next, from calibration to reporting, and every hand-over produced the expected file or table. "Functional pass" means the output was produced and checked, but no accuracy was measured.

6.4 System Testing

Table 6.3: System Testing

Steps	Test Data	Expected Results	Observed Results	Remarks
Step 1	Recorded intersection clip, end to end on the workstation GPU	System completes calibration, detection, tracking, violation logging, and reporting	The run completed and produced the annotated video, Excel report and dashboard	Pass
Step 2	Clip with partly faded lane markings	Automatic calibration warns when its geometry is weak, and manual lane correction is available	Manual lane addition was used to align the lane model with the visible road markings	Pass

Steps	Test Data	Expected Results	Observed Results	Remarks
Step 3	Clip with dense, mixed traffic (cars, autos, trucks)	Vehicle classes and counts appear in the outputs	Counts and class mix were produced; they were not compared with a manually annotated ground truth	Pass
Step 4	Clip containing vehicles near lane lines	Lane-straddling candidates flagged with lane, time and evidence image	Candidates and evidence images were produced and spot-checked on selected frames; precision and recall were not measured	Pass

System testing ran the whole pipeline on a recorded intersection clip, from upload to the final reports, and checked that each output was produced and complete. All of these tests passed, again as a functional check and not an accuracy measurement.

6.5 Functional Testing

Table 6.4: Functional Testing

Steps	Test Data	Expected Results	Observed Results	Remarks
Step 1	Upload a video through the Gradio interface	Video accepted and details (resolution, duration) shown correctly	Details were shown	Pass
Step 2	Start automatic calibration	Calibration completes and reports a geometry-consistency score and warnings	A calibration preview and geometry-consistency score were reported.	Pass
Step 3	Run full analysis on the calibrated video	Annotated output video produced with lane overlays and vehicle boxes	Annotated video was produced	Pass
Step 4	Download the Excel report	Report downloads and opens with the summary figures filled in	Report opened with the summary figures filled in	Pass
Step 5	Compare selected flagged violations with the annotated video and evidence images	Selected candidates correspond to visible vehicle behaviour	Selected candidates were inspected manually.	Pass
Step 6	Open the Review & Fines tab and approve a violation with an edited fine	Suggested fine is pre-filled and can be edited; approval is recorded	Suggested fine was pre-filled, the edit was accepted, and approval was recorded	Pass

The functional tests follow what an operator does in the interface: upload, calibrate, analyse, review violations and download the report. Each action behaved as expected. “Functional pass” here means the feature worked, but its accuracy was not measured.

CHAPTER 7

RESULTS AND DISCUSSION

The Phase I detector (`best.pt`, trained on the Indian Road User Vehicle Dataset) reached a mAP@0.5 of 87.4% on the validation split, with class-wise accuracy of 90.4% for cars and 89.5% for auto-rickshaws. These are validation-split figures, not results on an independent test set. The rest of this chapter walks through the output of one run of the full pipeline on a recorded clip. The clip was **99 seconds** long and showed mixed intersection traffic. It was processed with a frame stride of 2 on the NVIDIA RTX A2000 GPU. The run produced **90** tracks, of which **65** crossed the counting line and were counted, worth **72.8** PCU in total, with a mean speed of **21.6** km/h and an overall indicative LOS class of **C**. **26** tracked vehicles (**28.9%**) triggered at least one violation candidate, giving **30** candidates in total. All values in this chapter are taken from the Summary sheet and dashboard of this run and describe this one clip .

7.1 Interface Screens

Before the numbers, here are the main screens of the tool. The first screen is where the operator uploads the video and sees its resolution, duration, and frame rate. Calibration is a separate step so that the operator can check the lanes first. The interface provides a simple and user-friendly workflow for traffic analysis. It allows the operator to verify the video details and lane boundaries before processing. This helps reduce calibration errors and improves the reliability of the analysis process.

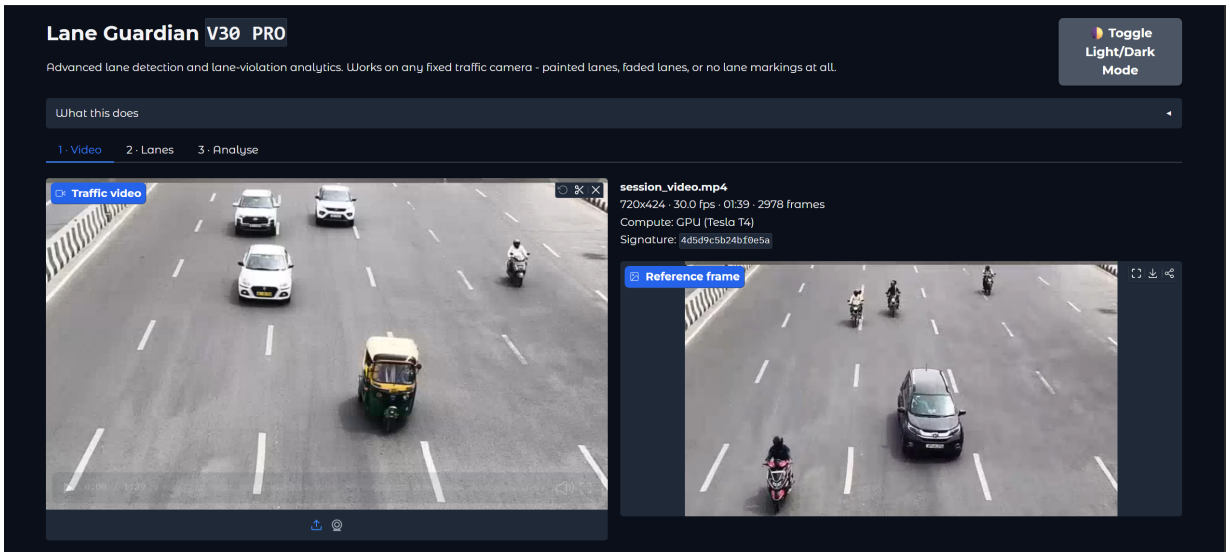


Figure 7.1: Gradio web interface: video upload and calibration engine selection

The road screen below shows the uploaded road video with the calibrated lane overlay on it. This is the view an operator uses to judge whether the lanes sit on the real road before starting the analysis.

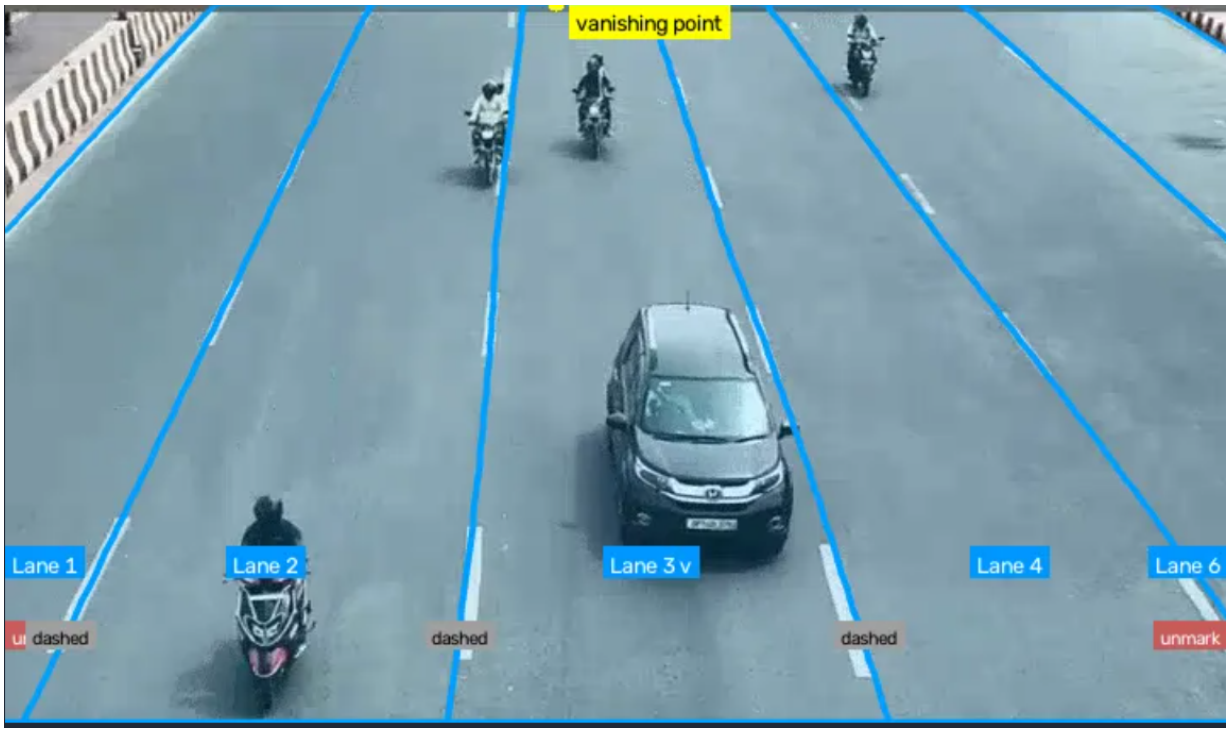


Figure 7.2: Road view with calibrated lane overlay before analysis

When the markings are faded or the automatic calibration is unreliable, the operator can add or correct lanes by hand on this screen, and merge them with the automatically detected lanes. The screen below shows a lane being added by hand and merged with the automatically detected lanes.

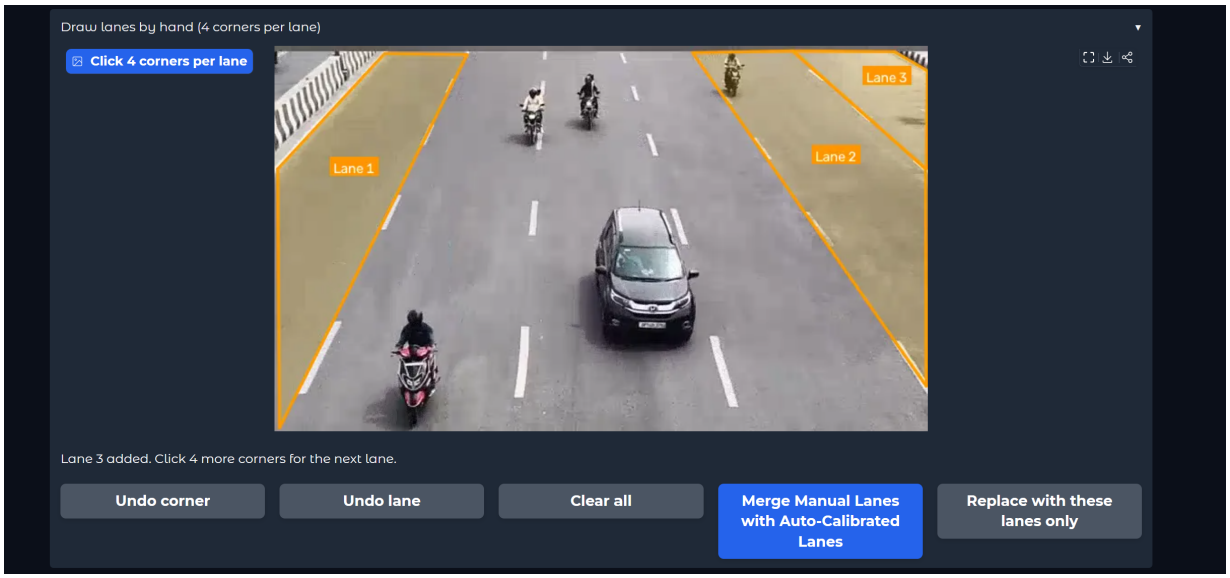


Figure 7.3: Manual lane addition screen

7.2 Calibration, Detection and Tracking

The automatic calibration produced the lane boundaries visible in the calibration preview. It also reported the estimated vanishing point, focal length, metric scale and a geometry-consistency score. The exact values for this run are stored in the Calibration sheet of the Excel report, so they are not repeated here. The score shows how well the detected geometric cues agree with each other. It is not a measure of lane-detection accuracy. Since the metric scale is approximate, the speeds are approximate too. Manual lane correction was used where needed to align the lane model with the visible markings, and the final merged lane model was used for all counts below.

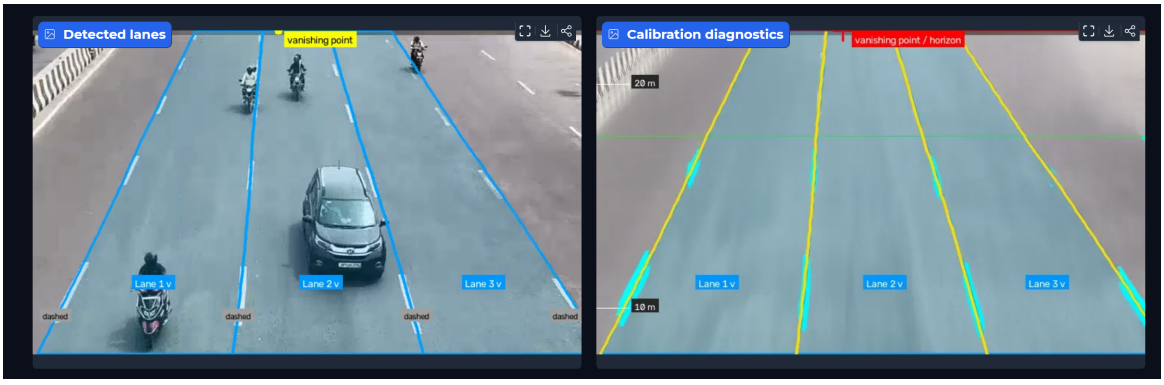


Figure 7.4: Calibration diagnostics – vanishing point and fitted lane boundaries

The annotated video shows each tracked vehicle with a class-coloured box, its track ID and the lane

polygons from the estimated geometry. A reviewer can use it to check whether a flagged candidate really is a vehicle crossing a line or just a false alarm from the rule engine.

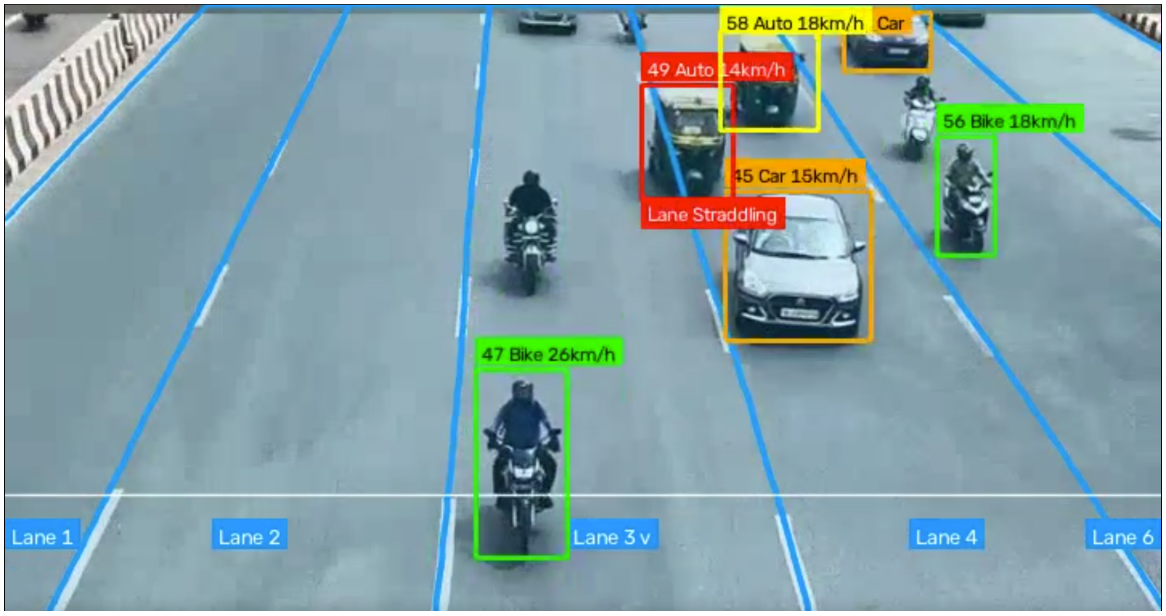


Figure 7.5: Annotated output frame with tracked vehicles and lane overlays

7.3 Flow Analysis

The lane analysis table gives, for each lane, the vehicles counted at the virtual line, PCU, flow in vehicles/hour and PCU/hour, observed peak 60-second flow, occupancy and indicative LOS class. Over all lanes the run counted **65** vehicles worth **72.8** PCU, and the busiest lane can be read from the figure below. The hourly values are the counts from a short clip scaled up to one hour, and the peak value is the highest count in any 60-second window, so both describe observed flow in this clip only. They are not saturation flow, which would need queue discharge headways over several signal cycles. The LOS class is a software-defined traffic-condition class based on the ratio of mean observed speed to the configured speed limit. It is an indicative project metric, not a Highway Capacity Manual LOS.

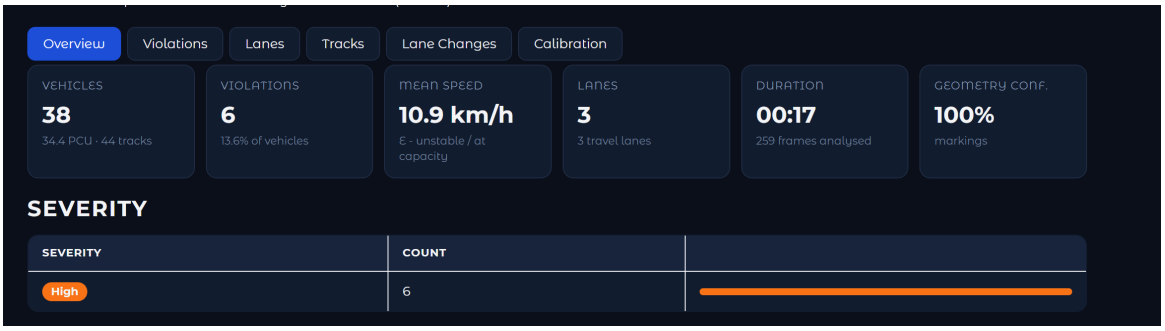


Figure 7.6: Lane-wise flow, PCU/hour, and Level of Service

The vehicle class chart shows each class's share of the counted vehicles and its contribution in PCU. In this run the counted vehicles were **49 cars (75.4%), 11 autos (16.9%) and 5 trucks (7.7%)**, worth 49, 8.8 and 15 PCU respectively, so the five trucks carried about 21% of the PCU load. A class with a small share of the count can carry a larger share of the PCU load, since a heavy vehicle weighs 3.0 against 1.0 for a car. The PCU weights used in the prototype (car 1.0, two-wheeler 0.5, auto-rickshaw 0.8, bus 3.0, truck 3.0) are implementation-level assumptions for comparing traffic load between lanes. A formal field study should use values from the selected traffic-engineering standard.



Figure 7.7: Vehicle class distribution and PCU contribution

7.4 Violations

The system logged **30** violation candidates in this run: Lane Straddling **22**, Unsafe Lane Change **5**, Overspeeding **2** and Unauthorized Parking **1**, with no Illegal Lane Change or Wrong-Way candidates in this clip. These come from rules and thresholds and were not checked against manually annotated ground truth, so the relative counts say nothing reliable about how common each violation is on

the road. The candidates may also depend on how clearly the markings are defined in different parts of the road, but that needs proper ground-truth evaluation before any relationship can be claimed.



VIOLATION TYPES

TYPE	COUNT	
Lane Straddling	6	

VEHICLE MIX

CLASS	COUNT	SHARE	PCU
Bike	20	52.6%	0.5
Auto	8	21.1%	0.8
Car	6	15.8%	1.0
Truck	3	7.9%	3.0
Bus	1	2.6%	3.0

Figure 7.8: Violation log – type, severity, lane, and confidence

For every logged violation the system saves a small crop of the vehicle at the moment it was flagged, together with a short description of the rule that fired (for example, how much of the vehicle was over the line and for how long). These crops make it easy to check by hand whether an event was a real violation or a borderline case whose threshold needs adjusting.



Figure 7.9: Evidence crop for a flagged Lane Straddling violation

The violation rules and their thresholds can be changed from the interface. The next screen shows that page, so the operator can see which limits (overlap fraction, time over the line, speed limit, parking duration, lane-change timing) produced the log above. The default parking duration is 30 seconds with a movement threshold of 1.2 m, and both are configurable.

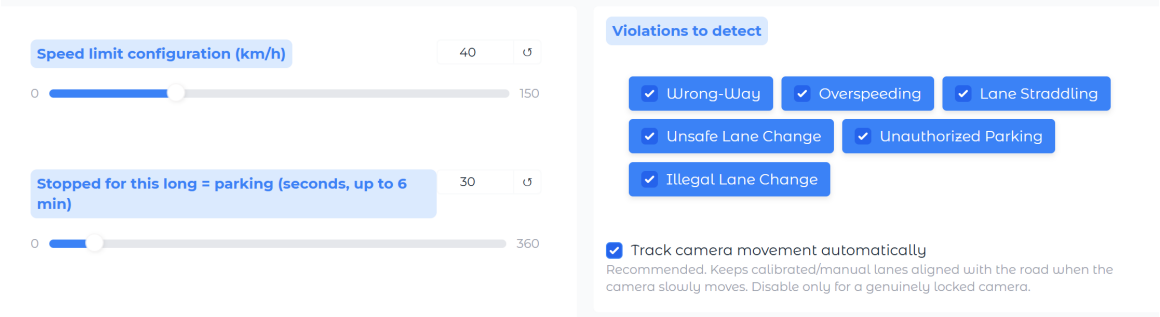


Figure 7.10: Violation logic and threshold settings page

The Review & Fines tab uses the same evidence crops but shows them one violation at a time, next to the vehicle's class, lane, speed and a suggested fine, so the operator does not have to cross-check a spreadsheet. The defaults are reference amounts for review (for example, Rs. 500 for Lane Straddling and Rs. 1000 for Unsafe Lane Change) and are not a legal determination of the penalty. Approving a card and saving writes the approval status and the possibly edited fine into a fresh copy of the Excel report, so the final report records the operator's decision and not just the rule output.



Figure 7.11: Review & Fines tab – evidence, suggested fine, and approval

7.5 Dashboard and Reports

The Dashboard sums the run up in a few cards: vehicles (with PCU and number of tracks), violations (with the share of vehicles flagged), mean speed with the indicative LOS class, number of lanes, duration, the geometry-consistency score and the observed peak 60-second flow. A non-technical reviewer can read the main numbers here without opening the Excel workbook.



Figure 7.12: Dashboard summary view

8	00:10	25	Car	Lane Straddling	High	0.34	Lane 2	22.6		32% of the vehicle over the line for 0.5s
9	00:15	36	Truck	Lane Straddling	High	0.65	Lane 4	12.6		49% of the vehicle over the line for 0.7s
10	00:16	40	Car	Lane Straddling	High	0.3	Lane 3	22.9		15% of the vehicle over the line for 0.5s
11	00:16	39	Auto	Lane Straddling	High	0.45	Lane 5	15		43% of the vehicle over the line for 0.5s
12	00:18	49	Auto	Lane Straddling	High	0.48	Lane 3	14		45% of the vehicle over the line for 0.5s

Figure 7.13: Excel report with the Summary and Violations sheets

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

LANE-GUARDIAN is an end-to-end prototype developed to analyse lane discipline and lane-wise traffic flow using recorded video from a fixed camera. The system estimates lane geometry using road markings, perspective information and vehicle trajectories, with manual correction when required. A custom-trained YOLOv10n model detects and classifies vehicles, while ByteTrack tracks their movements. A rule-based engine identifies potential lane discipline violations by checking vehicle paths against lane boundaries. The tracked vehicle counts are converted into PCU-based observed traffic flow, along with an indicative Level of Service (LOS) classification. The system generates an annotated video, an Excel report and a dashboard. It also allows an operator to review violation evidence and approve or reject violation candidates.

8.2 Future Scope

Future work will focus on improving the accuracy and reliability of LANE-GUARDIAN by evaluating its performance using annotated datasets and metrics such as precision, recall and F1-score. Accurate saturation-flow estimation can be introduced using queue-discharge headways across multiple signal cycles. Camera calibration, PCU weights and LOS classification can be validated against appropriate traffic-engineering standards. Further improvements include integrating live CCTV feeds, measuring processing latency and testing under different conditions, including night-time, rain and poor visibility. Evaluating the system at multiple intersections with diverse traffic conditions will help improve its robustness and practical usefulness for traffic monitoring and management.

REFERENCES

[1] S. Adhikary, N. Roy, and S. Jana, "Enhancing CCTV-based traffic violation detection using AI for automated enforcement," in Proc. Int. Conf. Comput. Intell. Appl. (CIACON), 2025, pp. 1–6.

[2] A. Sayar and Y. Dikilitas, "A real-time traffic violation detection system on highways using surveillance cameras and message-oriented middleware," IEEE Access, vol. 14, pp. 27546–27554, 2026.

[3] A. Ali, R. Sarkar, and D. K. Das, "IRUVD: A new still-image based dataset for automatic vehicle detection," Multimedia Tools and Applications, vol. 83, no. 3, pp. 6755–6781, 2024.

[4] M. Bakirci, "Utilizing YOLOv8 for enhanced traffic monitoring in intelligent transportation systems (ITS) applications," Digital Signal Processing, vol. 152, p. 1094, 2024.

[5] Q. Luo and Y. Qian, "Vehicle and pedestrian detection algorithm in foggy weather based on improved YOLO v10," in Proc. 6th Int. Conf. Artif. Intell. Electromech. Autom. (AIEA), 2025, pp. 291–295.

[6] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-time end-to-end object detection," Advances in Neural Information Processing Systems (NeurIPS), arXiv:2405.14458, 2024.

[7] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in Proc. IEEE Int. Conf. Image Process. (ICIP), 2016, pp. 3464–3468.

[8] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in Proc. Eur. Conf. Comput. Vis. (ECCV), 2022, pp. 1–21.

[9] M. V. Yashina, Y. A. Akilin, V. V. Osada, and A. F. Smyk, "Vehicle tracking algorithm of speed estimation based on traffic video analysis," in Proc. Intell. Technol. Electron. Devices Veh. Road Transp. Complex (TIRVED), 2024, pp. 1–6.

[10] G. S. P. K. Segu, “Real time road lane detection and vehicle detection on YOLOv8 with interactive deployment,” in Proc. 16th IEEE Int. Conf. Comput. Intell. Commun. Netw. (CICN), 2024, p. 267.

[11] U. Sakthi and S. Badoni, “Real-time lane detection and vehicle speed prediction for intelligent transportation systems,” SRM Institute of Science and Technology, Kattankulathur, India, 2024.

[12] S. Gowroju, A. Banu, S. Choudhary, E. V. Kumar, N. Sathwik, and N. Ashritha, “UrbanEye: Deep learning enhanced two-wheeler traffic rule violation detection using YOLO and OCR,” in Proc. Int. Conf. Innov. Intell. Syst. (ISAC3), 2025, pp. 1–5.

[13] L. Selvam, “Efficient traffic rule enforcement framework using YOLOv8-based object detection,” SRM Madurai College for Engineering and Technology, Pottapalayam, India, 2024.

[14] Y. M. Bhavsar, M. S. Zaveri, M. S. Raval, and S. B. Zaveri, “Vision-based investigation of road traffic and violations at urban roundabout in India using UAV video: A case study,” Transportation Engineering, vol. 14, p. 100207, 2023.

[15] A. Patil, B. Chilad, S. Jadhav, and A. Tigadi, “Multi camera vehicle tracking using OpenCV & deep learning,” in Proc. 2nd Int. Conf. Futuristic Technol. (INCOFT), 2023, pp. 1–5.

[16] K. Soma, L. Shibu, and M. N., “A real-time vehicle detection and speed estimation using YOLO V8,” in Proc. Int. Conf. Adv. Data Eng. Intell. Comput. Syst. (ADICS), 2024, pp. 1–6.

[17] M. Haloi and D. B. Jayagopi, “A robust lane detection and departure warning system,” arXiv:1504.07590, 2015.

[18] S. B. Wicaksono, M. Hamdani, A. Yasar, and L. Li, “A survey on traffic violations prediction with deep learning,” Transp. Res. Procedia, vol. 91, pp. 219–226, 2025.

[19] K. Klubsuwan, W. Koodtalang, and S. Mungsing, “Traffic violation detection using multiple trajectories evaluation of vehicles,” in Proc. 4th Int. Conf. Intell. Syst. Model. Simul. (ISMS), 2013, pp. 220–224.

[20] A. Karim, M. A. Raza, Y. Z. Alharthi, G. Abbas, S. Othmen, M. S. Hossain, et al., "Visual detection of traffic incident through automatic monitoring of vehicle activities," World Electr. Veh. J., vol. 15, no. 9, p. 382, 2024.

[21] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 779–788.

[22] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," arXiv:2004.10934, 2020.

[23] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in Proc. IEEE Int. Conf. Image Process. (ICIP), 2017, pp. 3645–3649.

[24] M. Aly, "Real time detection of lane markers in urban streets," in Proc. IEEE Intell. Veh. Symp. (IV), 2008, pp. 7–12.

[25] D. Neven, B. De Brabandere, S. Georgoulis, M. Proesmans, and L. Van Gool, "Towards end-to-end lane detection: An instance segmentation approach," in Proc. IEEE Intell. Veh. Symp. (IV), 2018, pp. 286–291.

[26] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," Commun. ACM, vol. 24, no. 6, pp. 381–395, 1981.

[27] R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[28] Z. Zhang, "A flexible new technique for camera calibration," IEEE Trans. Pattern Anal. Mach. Intell., vol. 22, no. 11, pp. 1330–1334, 2000.

[29] R. E. Kalman, "A new approach to linear filtering and prediction problems," Trans. ASME J. Basic Eng., vol. 82, no. 1, pp. 35–45, 1960.

[30] Transportation Research Board, Highway Capacity Manual 6th Edition: A Guide for Multimodal Mobility Analysis. Washington, DC, USA: Transportation Research Board, 2016.

APPENDIX

i) Plagiarism Report

ii) Conference/Journal Paper and Certificate

iii) Project Exhibition Certificate